

Performance Analysis of Sorting Algorithms

Student Name: Ajaya Kumar Patel

Roll Number: CRS2503

Course: Cryptographic and Security Implementations

August 3, 2026

Contents

1	Introduction	2
2	Experimental Setup	2
3	Analysis 1: Normalized Comparisons and Swaps	2
4	Analysis 2: CPU Performance Measurement	3
5	Analysis 3: Size vs Time Analysis	3
6	Conclusion	4
7	References	5

1 Introduction

This report presents the performance comparison of three sorting algorithms: Merge Sort, Quick Sort, and Heap Sort.

The algorithms were implemented in C and tested using randomly generated integer arrays. The comparison is based on operation count, CPU performance, and execution time for different input sizes.

2 Experimental Setup

The experiment was performed using:

- Array size: 10000 elements
- Number of iterations: 100
- Input data: Randomly generated integers
- CPU measurement: CPU cycle counter using `_rdtscp()`
- Time measurement: `clock_gettime()`

3 Analysis 1: Normalized Comparisons and Swaps

To compare the number of operations independent of input size, the number of comparisons and swaps were normalized using:

$$\frac{\text{Operation Count}}{n \log_2 n}$$

The results are shown below.

Algorithm	Normalized Comparisons	Normalized Swaps
Merge Sort	0.9069	0.0000
Quick Sort	1.2280	0.6834
Heap Sort	1.7716	0.9351

Table 1: Normalized comparison and swap count

Merge Sort performs only comparisons because it does not use swapping during merging. Quick Sort and Heap Sort perform swaps during rearrangement of elements.

4 Analysis 2: CPU Performance Measurement

The execution performance was measured using average CPU time, average CPU cycles, cycles per byte, and cycles per bit. The measurements were performed on randomly generated arrays of size $n = 10000$ elements, with 100 iterations to obtain the average values.

Algorithm	Average Time (ns)	CPU Cycles	Cycles/Byte	Cycles/Bit
Merge Sort	3479479.40	8684824.90	217.1206	27.140078
Quick Sort	1459799.91	3643670.24	91.0918	11.386470
Heap Sort	2079467.91	5190382.20	129.7596	16.219944

Table 2: CPU performance comparison

From the measurements, Quick Sort required the lowest execution time and CPU cycles for the tested random input. Heap Sort performed better than Merge Sort in execution time but required more operations than Quick Sort.

5 Analysis 3: Size vs Time Analysis

The execution time was measured for different input sizes ranging from 1000 to 10000 elements.

Array Size	Merge Sort	Quick Sort	Heap Sort
1000	275417.88	195645.10	350267.46
2000	664116.42	521876.46	958477.66
3000	1072025.18	894231.20	1612195.38
4000	1557009.86	1299957.92	2204984.48
5000	1942373.24	1602198.46	2900499.02
6000	2390179.00	1790800.76	2987531.72
7000	2572296.44	2163608.72	3857937.44
8000	2885401.46	2371364.62	4219053.52
9000	3398890.60	2354813.58	4344767.90
10000	3629935.42	2652794.62	4726660.78

Table 3: Execution time for different input sizes

The following graph shows the change in execution time with increasing array size.

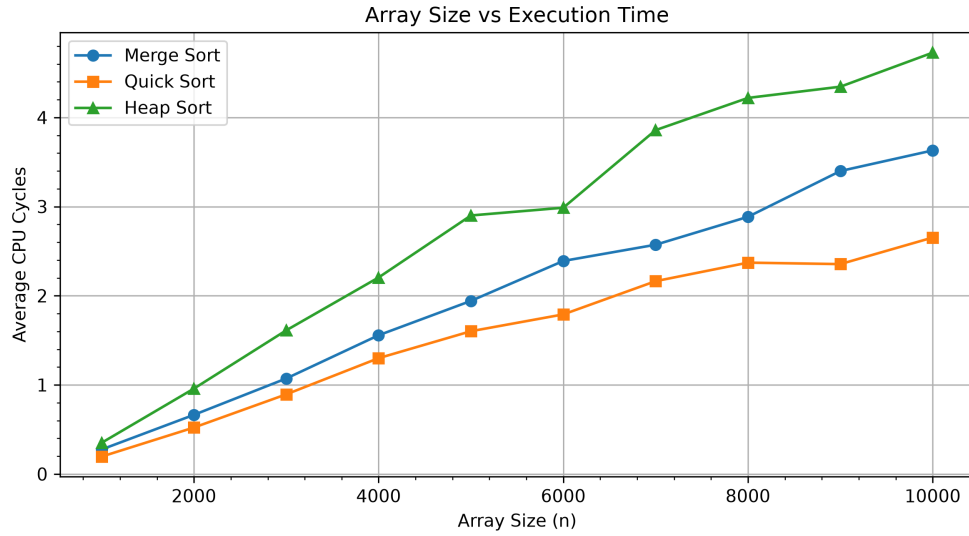


Figure 1: Array Size vs Execution Time

6 Conclusion

The experiment shows that all three algorithms follow approximately $O(n \log n)$ behavior.

For the tested random input:

- Quick Sort achieved the lowest execution time and CPU cycles.
- Heap Sort required more comparisons and swaps than Quick Sort.
- Merge Sort showed higher execution time because of additional merging operations and memory usage.

Overall, Quick Sort provided the best performance for the tested dataset.

7 References

1. OpenAI. *ChatGPT (GPT-5.5)*. Used for benchmarking methodology, implementation guidance, and code review. <https://chatgpt.com/>
2. GeeksforGeeks. *Merge Sort*. <https://www.geeksforgeeks.org/merge-sort/>
3. GeeksforGeeks. *Quick Sort*. <https://www.geeksforgeeks.org/quick-sort/>
4. GeeksforGeeks. *Heap Sort*. <https://www.geeksforgeeks.org/heap-sort/>